Case Study: Bespoke Labs Multi-Turn Tool Use

How a 7B model learned to chain tool calls with GRPO and one simple reward

Learning outcomes

After this case study, you can:

1. Describe the BFCL multi-turn RL environment and its grader.
2. Explain each GRPO setting in the final recipe.
3. Explain the length blow-up failure and how they fixed it.
4. Argue why one binary reward beat a three-part reward.
5. Design a similar RL environment for an Indian service.

The problem: agents must chain many tools

Prompting	Easy to start. Edge cases multiply, and each needs a new hand-written rule.
Fine-tuning	Needs demonstrations from people or a teacher model. These cost money, and the teacher limits quality.
RL	The model learns from its own attempts. It scales with compute, not with labels.

In April 2025, most open RL work was still single-turn.

The RL environment at a glance

Task set	100 training tasks from BFCL v3, multi-turn base.
Harness	Simulated Python APIs. The first user request only. Up to 10 tool calls.
State	The objects behind the APIs: bookings, balances, files, messages.
Grader	1 if BFCL’s state check and response check both pass. Else 0.

BFCL v3: eight simulated domains

File system
`ls, cd, cat`

Vehicle control
`startEngine, displayCarStatus`

Messages
`send_message, delete_message`

Support tickets
`create_ticket, close_ticket`

Travel booking
`book_flight, get_flight_cost`

Trading
`get_stock_info, place_order`

Social posts
`post_tweet, retweet`

Math
`logarithm, mean`

200 base multi-turn tasks. Experts wrote and checked each answer path.

One task, before and after training

Task: book a business-class flight from Los Angeles to New York on the morning of 15 April 2024. Use the linked card.

Before (step 52)

`book_flight`

Airport “NYC”, cost 0.

Reward: 0

After training

```
get_nearest_airport_by_city
get_flight_cost
book_flight
```

Reward: 1

The grader: two checks, one bit

State check

Compare the backend state with the expected state. This catches wrong writes and deletes.

Response check

The model’s calls must include the minimal correct calls. Extra calls are allowed.

Reward = 1 only if both checks pass. Otherwise 0.

The split: 100 to train, 100 held out

200
BFCL v3 multi-turn base tasks

100
for training

100
held out for testing

The model never trains on a test task. So the test score measures skill, not memory.

They used only the first user request of each task, to make training faster.

Training setup: model and compute

Setting	Value
Model	Qwen2.5-7B-Instruct
GPUs	4 × H200: 3 train, 1 runs vLLM
Length of run	1,600 steps (100 epochs)
Learning rate	1e-6, constant, 20 warm-up steps
Gradient clip	Max norm 0.2

Training setup: the GRPO settings

```
group size G = 8
μ (updates per batch) = 2
KL weight β = 0.001
reference model sync = every 100 steps
overlong filtering = on
max completion = 2,048 tokens
```

μ = 2 means one on-policy step and one off-policy step for each batch.

The result on the held-out tasks

55%

test accuracy before RL

78%

test accuracy after RL

100

training tasks, no demonstrations

Training stayed stable with the final recipe.

The main failure

Length blow-up

Outputs grew longer at each step.

Then they turned into gibberish: random characters and repeated tokens.

Then training and test accuracy collapsed.

What they tried against the blow-up

Attempt	Outcome
Dr. GRPO normalization	Still blew up
Gibberish penalty: −1 from a judge	Blew up sooner
Overlong filtering, no KL	Collapsed near step 300
Overlong filtering, β = 0.04	Collapsed
Overlong filtering, β = 0.001	**Stable: final recipe**

Two more ablations

Reward design

Three parts (tool runs, format, correctness) were less stable.

Correctness alone was best.

Reference model

A fixed reference model did worse.

A refresh every 100 steps did better.

The team thinks the extra reward parts invited reward hacking.

Five design rules from this case

1. Start with one binary reward for correctness.
2. Keep a small KL term. Do not set it to zero.
3. Mask over-long rollouts. Do not punish them with a judge.
4. Refresh the reference model as the policy improves.
5. Hold out test tasks. Watch output length to catch a blow-up early.

Reproduce it

Code: the bespokelabsai/verifiers repository, file verifiers/examples/bfcl_agent.py.

Compute: the original run used 4 H200 GPUs. Smaller GPUs need a smaller model.

```
accelerate launch
--config-file configs/zero3.yaml
--num-processes 3
verifiers/examples/bfcl_agent.py
```

Read the script before you run it. Some settings sit inside a test block.

Build an Indian version

Domain	Train tickets and UPI refunds, with synthetic users and accounts.
Tools	`search_trains, book_ticket, cancel_ticket, get_refund_status`
Requests	Written in Hinglish and in English, for the same tasks.
Grader	The final booking state, plus the required lookup calls.
Safety	Synthetic data only. No real PNR, Aadhaar or bank data.

Questions for discussion

1. Why was a three-part reward less stable than correctness alone?
2. Why did a –1 gibberish penalty make the blow-up come sooner?
3. 100 epochs on 100 tasks: what can the model learn, and what can it miss?
4. How would you extend this RL environment to every user turn?

Read the originals

- Bespoke Labs: Improving Multi-Turn Tool Use with RL (Apr 2025)
- Training code: bespokelabsai/verifiers
- BFCL v3 multi-turn benchmark (Berkeley, 2024)
- TWIML podcast 731 with Mahesh Sathiamoorthy (May 2025)
- DAPO: overlong filtering
- Dr. GRPO: length normalization